LAB University of Applied Sciences

Task-IV

UI testing

LAB University of Applied Sciences
Bachelor of Engineering Industrial Information Technology
2027
Davin Pihlajasaari

Statement about using AI

The author of this thesis, Davin Pihlajasaari, used the following AI tools during the preparation of the thesis:

ChatGPT

Purpose of use: Generating the code and commenting on edits done to it.

Explanation of the use of the tool: Used ChatGPT to generate a generic test suite and after making edits had it comment the code for me.

Responsibility

The author of this thesis, Davin Pihlajasaari, takes full responsibility for the content of this thesis and has reviewed and edited the content generated by the possible use of AI tools. The author has saved all discussions with AI applications during the thesis process. Thus, the extent of the author's own work and the use of AI can be verified if necessary.

The authenticity of this thesis has been checked with the Turnitin similarity checking software.

Abstract

Author(s)	Type of publication	Published in (year)
Davin Pihlajasaari	report	2026
	Number of pages	
	13	

Title of the publication

Task-IV

UI testing

Bachelor of Engineering Industrial Information Technology

Petri Rantanen

Abstract

A report on task four, a automated UI testing task.

Keywords

Selenium, UI, automation,

Contents

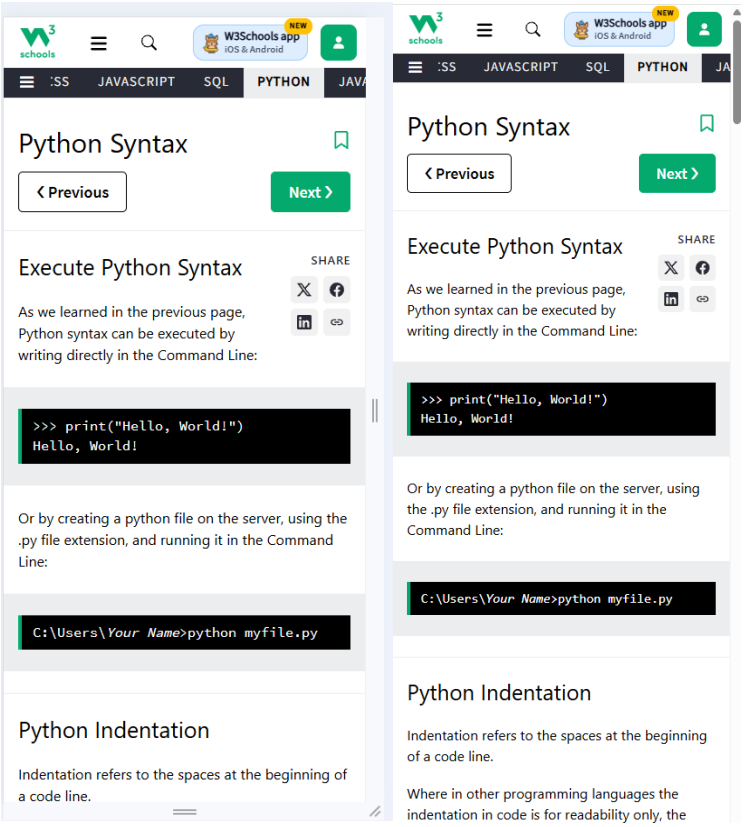
1 Manual testing 1
2 Automated testing 3
3 Comparison of techniques 4
References 5

Appendix 1. Code

1 Manual testing

The website tested in the manual testing stage was w3schools.com.

When set to mimic mobile screens it automatically transferred to a mobile version of the webpage, while when the screen size changed all pertinent elements shuffled into their correct places for that screen size.



figs 1. and 2. Fig one is the mobile version of the webpage and fig two is the compressed desktop version of the webpage.

The page layouts have barely any differences between them, so the page designers did a very good job in making these webpages.

Next, I did some CPU slowdown and network slowdowns and got some results.

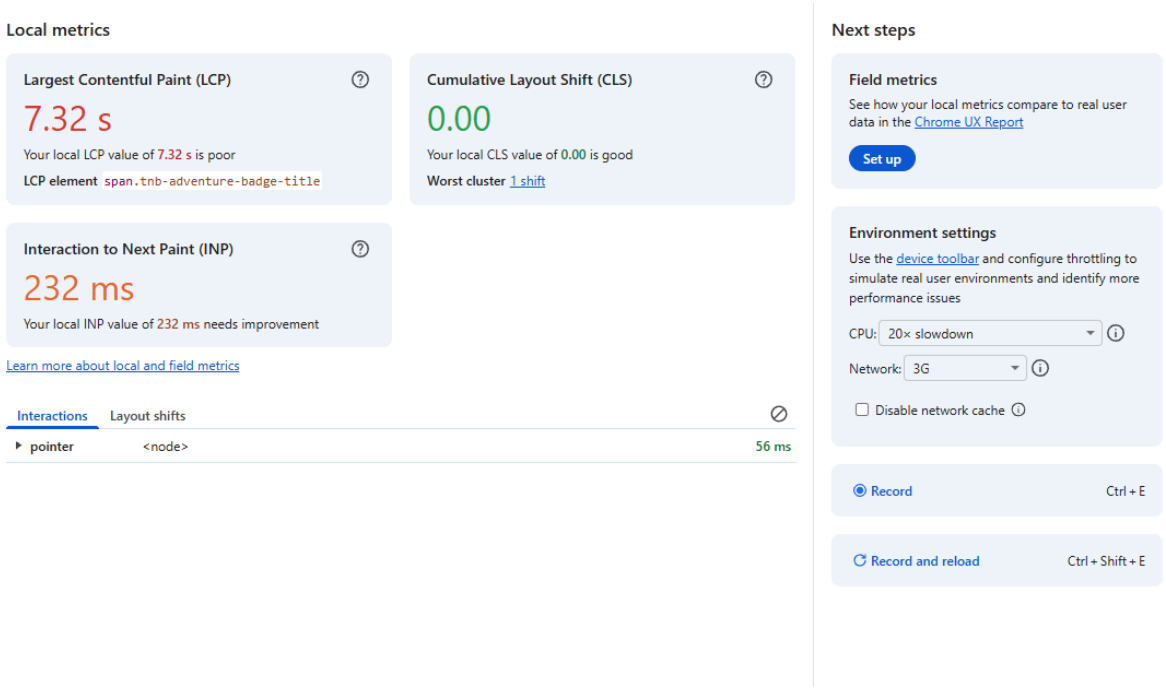


Fig 3. This image shows the response times with 20x CPU slowdown and 3G network speeds.

The above figure shows that it took nearly seven and a half seconds to render the largest item on the page and took around a quarter of a second to react to being interacted with. This test was performed by clicking on a next button to go to the next lesson in this category. Compared to no restrictions on CPU and network with the same test.

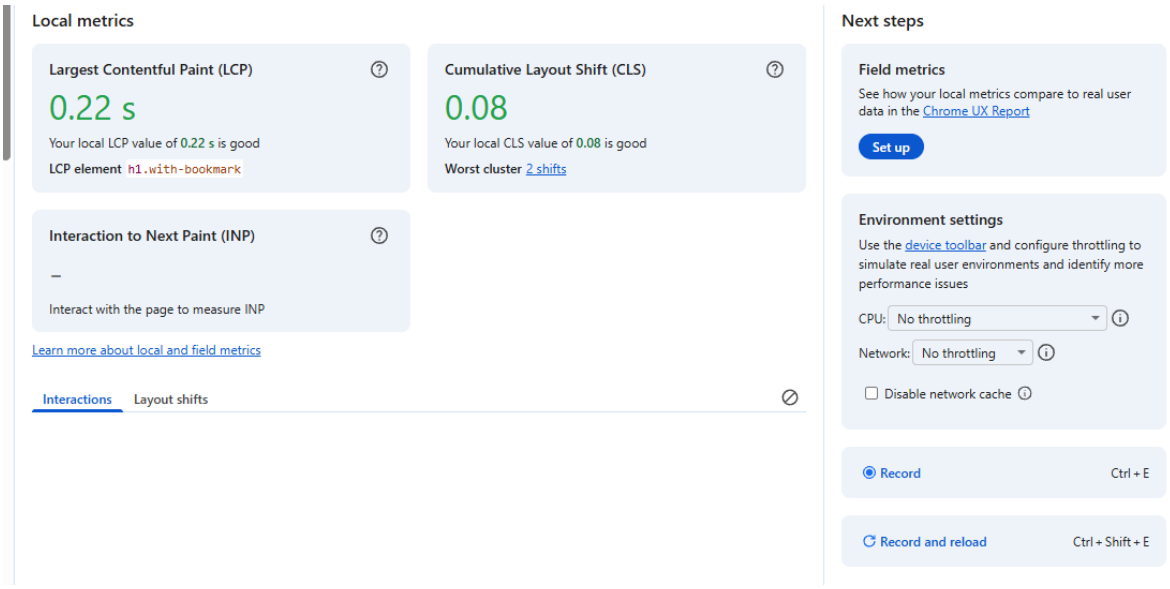


Fig 4. This image shows the response time without restrictions. Although it is not showing the INP was around 5-20 miliseconds.

2 Automated testing

Instead of using the automated testing suite provided during the lesson I had ChatGPT make a new generic test suite and tested it and then got help making edits as I familiarized myself with selenium. My test suite checks for these basic page requirements:

test_page_loads	PASSED
test_page_has_body	PASSED
test_page_has_no_horizontal_overflow	PASSED
test_images_load	PASSED
test_links_have_destinations	PASSED
test_buttons_are_interactable	FAILED
test_no_broken_internal_links	PASSED
test_page_has_no_horizontal_over-flow_after_resize	PASSED

The reason the button test failed is probably because there are no universal cookies accept standard and I could not make it work properly. So, the cookies banner just blocked all other buttons while it was on top of the page.

- The first test checks if the page loads properly and has a valid url.
- The second test checks if the body is visible.
- The third checks to see if there is any horizontal overflow.
- The fourth checks to see if all images load properly.
- The fifth checks all visible links to see if they have a valid destination.
- The sixth which fails checks to see if visible buttons and controls are enabled.
- The seventh visits internal links and verifies that they work.
- The eighth one checks for overflow in tablet and mobile configurations.

3 Comparison of techniques

I feel that the manual testing is better suited for testing out UI layout visually and for accessibility testing, to see how users feel while using the website and the automated testing is better suited for testing site performance and looking for small bugs and errors that get past the basic manual checks.

The advantage of the manual approach is that you can get a feel for how the page responds to a person and how they navigate it, to see if the button positioning is unintuitive for a basic user.

The disadvantages are that you can’t do a super in-depth test manually.

The advantages of the automatic testing are that you can-do in-depth testing on all connected pages and see if there are any formatting errors and the like.

The disadvantages are that they don’t give you an organic report it is only technical outputs.

References

Link to GitHub repo:
https://github.com/DavinPihlajasaari/Devops_engineering/tree/main/Petri/task-IV-UI-testing

Appendix 1.

```
import pytest
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

BASE_URL = "https://www.w3schools.com"
TIMEOUT = 10

@pytest.fixture
def driver():
    """Create a Chrome browser and close it after each test."""
    options = webdriver.ChromeOptions()
    options.add_argument("--window-size=1920,1080")
    browser = webdriver.Chrome(options=options)
    browser.implicitly_wait(2)
    yield browser
    browser.quit()

def accept_cookies(driver):
    """Accept W3Schools consent through the actual form control."""
    try:
        accept_button = WebDriverWait(driver, TIMEOUT).until(
            EC.element_to_be_clickable((
                By.CSS_SELECTOR,
                "form#fast-cmp-form "
                "article#fast-cmp-home "
                "button.fast-cmp-button-primary[value='all']"
            ))
        )

        accept_button.click()

        WebDriverWait(driver, TIMEOUT).until(
            EC.invisibility_of_element_located(
                (By.ID, "fast-cmp-home")
            )
        )
```

```
            print("Cookie consent accepted.")

        except Exception as error:
            dialog = driver.find_elements(By.ID, "fast-cmp-home")

            if not dialog or not dialog[0].is_displayed():
                print("Cookie consent dialog not present.")
            else:
                raise RuntimeError(
                    "Could not accept W3Schools cookie consent."
                ) from error

@pytest.fixture
def page(driver):
    """Open the website and handle its cookie banner before testing."""
    driver.get(BASE_URL)
    accept_cookies(driver)
    return driver


def test_page_loads(page):
    """Check that the page loads with a valid URL and non-empty title."""
    assert page.current_url.startswith("http")
    assert page.title.strip() != ""


def test_page_has_body(page):
    """Check that the page body becomes visible."""
    body = WebDriverWait(page, TIMEOUT).until(
        EC.visibility_of_element_located((By.TAG_NAME, "body"))
    )
    assert body.is_displayed()


def test_page_has_no_horizontal_overflow(page):
    """Check that the page does not overflow horizontally."""
    has_overflow = page.execute_script(
        "return document.documentElement.scrollWidth > "
        "document.documentElement.clientWidth;"
    )
    assert not has_overflow, "Page has horizontal overflow"


def test_images_load(page):
```

```
    """Check that every visible image has a source and loads successfully."""
    images = page.find_elements(By.TAG_NAME, "img")

    for image in images:
        if image.is_displayed():
            src = image.get_attribute("src")
            assert src, "Image has no source"
            assert image.get_property("complete") is True, (
                f"Image not fully loaded: {src}"
            )
            assert image.get_property("naturalWidth") > 0, (
                f"Image broken or 0px wide: {src}"
            )

def test_links_have_destinations(page):
    """Check that every visible link has a destination URL."""
    links = page.find_elements(By.TAG_NAME, "a")

    for link in links:
        if link.is_displayed():
            assert link.get_attribute("href"), "Link has no destination"

def test_buttons_are_interactable(page):
    """Check that visible buttons and submit controls are enabled."""
    buttons = page.find_elements(
        By.CSS_SELECTOR,
        "button, input[type='button'], input[type='submit']"
    )

    for button in buttons:
        if button.is_displayed():
            assert button.is_enabled(), (
                f"Button is unexpectedly disabled: {button.text}"
            )

def test_no_broken_internal_links(page):
    """Visit internal links and verify that each page reaches a ready state."""
    base_origin = page.execute_script("return window.location.origin")
    links = page.find_elements(By.TAG_NAME, "a")

    internal_urls = {
```

```
        link.get_attribute("href")
        for link in links
        if link.get_attribute("href")
        and link.get_attribute("href").startswith(base_origin)
    }

    for url in internal_urls:
        page.get(url)
        WebDriverWait(page, TIMEOUT).until(
            lambda browser: browser.execute_script(
                "return document.readyState"
            ) in ("interactive", "complete")
        )

    # Restore the starting page for consistency.
    page.get(BASE_URL)
    accept_cookies(page)

def test_page_has_no_horizontal_overflow_after_resize(page):
    """Check horizontal overflow at desktop, tablet, and mobile widths."""
    for width in (1920, 1366, 768, 375):
        page.set_window_size(width, 900)

        WebDriverWait(page, TIMEOUT).until(
            EC.visibility_of_element_located((By.TAG_NAME, "body"))
        )

        has_overflow = page.execute_script(
            "return document.documentElement.scrollWidth > "
            "document.documentElement.clientWidth;"
        )

        assert not has_overflow, (
            f"Horizontal overflow at width {width}px"
        )
```